

Universidad Autónoma de Baja California



Facultad de Ingeniería, Mexicali

Materia: Microcontroladores

“Actividad 4.7: Motor PWM”

Docente: Carlos Alberto Aguilar Avelar

Alumnos:

1184754 Jesus Alejandro Manjarrez Torres

1185117 Juan Pablo Green Marcial

Fecha: 26/04/2026

INTRODUCCIÓN

En esta actividad de microcontroladores se describe el diseño y la simulación de un regulador de velocidad para un motor de corriente directa utilizando el microcontrolador **ATmega328P**. El objetivo principal es aplicar los conocimientos sobre temporizadores (Timers) y generación de señales **PWM** para manipular la velocidad del motor en cinco niveles distintos (0%, 25%, 50%, 75% y 100%), seleccionables mediante interruptores. Para lograr un control bidireccional y una gestión adecuada de la potencia, se emplea el circuito integrado **L293D**, el cual permite separar la etapa de control de baja potencia de la etapa de fuerza necesaria para el motor de 12V. A través del entorno **Proteus**, se verifica la correcta respuesta del sistema ante las entradas del usuario, garantizando una transición fluida entre las velocidades y una inversión de marcha precisa, cumpliendo así con los requisitos técnicos de la práctica de microcontroladores, como se mencionó anteriormente no se implemento nada fisico, solamente fue la pura simulación en Proteus.

DESARROLLO

Código utilizado para lograr las señales PWM

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>

void init_pwm() {
    // Configurar PD6 (OC0A) como salida
    DDRD |= (1 << DDD6);

    // Configurar Timer 0: Fast PWM, modo no invertido
    TCCR0A = (1 << COM0A1) | (1 << WGM01) | (1 << WGM00);

    // Prescalador de 64
    TCCR0B = (1 << CS01) | (1 << CS00);
}

void init_io() {
    // PD0 y PD1 como salidas para el driver (D+ y D-)
    DDRD |= (1 << DDD0) | (1 << DDD1);

    // PB0 hasta PB4 como entradas
    DDRB &= ~(1 << DDB0) | (1 << DDB1) | (1 << DDB2) | (1 << DDB3) | (1 <<
DDB4));

    // Activar resistencias PULL-UP internas en Puerto B
    PORTB |= (1 << PORTB0) | (1 << PORTB1) | (1 << PORTB2) | (1 << PORTB3) |
(1 << PORTB4);
}

int main(void) {
    init_pwm();
    init_io();

    while (1) {

        // Si el switch está cerrado, el pin lee 0 (LOW)
        if (!(PINB & (1 << PINB0))) {
            PORTD |= (1 << PORTD0);
            PORTD &= ~(1 << PORTD1);
        } else {
            PORTD &= ~(1 << PORTD0);
            PORTD |= (1 << PORTD1);
        }

        //!(PINB & ...) es verdadero si el switch se presiona
        if      (!(PINB & (1 << PINB4))) OCR0A = 255; // 100%
        else if (!(PINB & (1 << PINB3))) OCR0A = 191; // 75%
        else if (!(PINB & (1 << PINB2))) OCR0A = 127; // 50%
```

```

        else if (!(PINB & (1 << PINB1))) OCR0A = 64; // 25%
        else OCR0A = 0; // 0% (Ninguno
presionado)

        _delay_ms(50);
    }
}

```

Se utiliza el **ATmega328P** el cual se encarga de gestionar las entradas y generar las señales de control necesarias, debido a que el motor de corriente directa requiere una alimentación de **12V** y una demanda de corriente que excede las capacidades de los pines de salida del microcontrolador, se integró el driver **L293D** como interfaz de potencia. Este dispositivo permite manejar la carga de fuerza de manera segura, utilizando su configuración interna de puente H para facilitar tanto la regulación de velocidad como la inversión del sentido de giro.

Para lograr un control preciso sobre las revoluciones del motor, se configuró el periférico **Timer 0** del microcontrolador en modo **Fast PWM** (Modulación por Ancho de Pulso Rápida). Esta señal de salida se emite a través del pin **PD6**, identificado también como **OC0A**. La frecuencia de conmutación se estableció mediante un prescalador de 64, lo que resulta en una frecuencia de aproximadamente **976.5Hz** considerando un reloj de sistema de **16MHz**. Esta elección técnica es fundamental para asegurar una respuesta fluida del motor y minimizar el ruido audible generado por la conmutación de los transistores internos del driver durante la operación a diferentes ciclos de trabajo. El desarrollo del firmware en lenguaje C se centró en la creación de un bucle infinito que monitorea constantemente el estado del Puerto B, lugar donde se conectaron los interruptores deslizables. Al activar las resistencias *pull-up* internas del ATmega328P, se simplificó el circuito físico y se estableció una lógica negativa para la detección de eventos. Dependiendo de cuál de los cinco interruptores de velocidad sea accionado, el programa actualiza el registro **OCR0A** con valores proporcionales al **25%**, **50%**, **75%** o **100%** del ciclo de trabajo, también otro interruptor determina la polaridad de las salidas digitales **PD0**, definiendo así el sentido de giro del motor de forma inmediata.

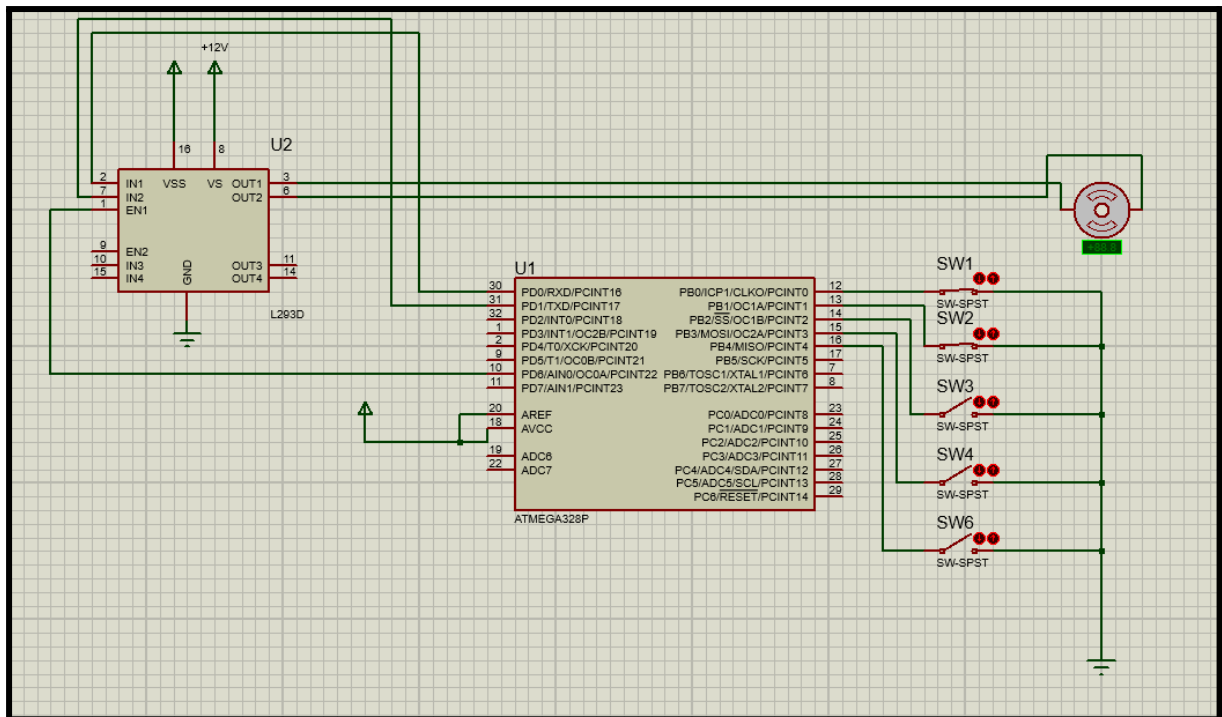


Diagrama Simulado en Proteus.

Link con la demostración del funcionamiento:

<https://youtu.be/RvCwoMLndC0?si=FRbD6KJSJrcpnEWr4>

CONCLUSIÓN

En conclusión, el desarrollo de esta actividad permitió validar la eficiencia de la técnica de **Modulación por Ancho de Pulso (PWM)** como el método estándar para el control de velocidad en motores de corriente directa. A través de la configuración de los registros del **ATmega328P**, se demostró que es posible gestionar la entrega de potencia de manera precisa y eficiente, evitando el desperdicio de energía en forma de calor que ocurriría con métodos de regulación lineal. La integración del driver **L293D** resultó fundamental, ya que no solo proveyó el aislamiento necesario entre la lógica del microcontrolador y la etapa de fuerza, sino que también facilitó el control bidireccional mediante su estructura de puente H, permitiendo una respuesta inmediata a las demandas del usuario. La práctica nos muestra la importancia de las resistencias de **pull-up** (o pull-down) y la correcta gestión de los estados lógicos para evitar comportamientos erráticos en el sistema. Al utilizar las resistencias internas del microcontrolador y diseñar una lógica de software que prioriza las entradas de los interruptores, se logró un sistema de control funcional y libre de "ruido" digital. Este enfoque simplifica el hardware necesario sin comprometer la fiabilidad del control.

REFERENCIAS

- https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf
- **Labcenter Electronics.** (2024). *Proteus VSM Simulation: Microcontroller Simulation Reference Guide.*